

Enterprise Knowledge Assistant

System Design Document

1. Introduction

The Enterprise Knowledge Assistant is a local-first Retrieval Augmented Generation (RAG) system designed to answer questions from internal enterprise documentation using natural language. The system is intended for organizations that need a practical, privacy-preserving, and technically robust way to make internal knowledge searchable and accessible to employees.

The solution combines document ingestion, semantic retrieval, context-grounded generation, and source citation into a single end-to-end workflow. Unlike many cloud-based assistants, this implementation runs locally using Ollama, allowing sensitive internal documents to remain on-device while still delivering high-quality conversational responses.

This document presents the architecture, design rationale, data flow, retrieval strategy, evaluation approach, and scalability considerations of the system.

2. Problem Statement

Enterprises often store critical knowledge across multiple document types, including HR policies, technical guides, customer support content, compliance documents, and process handbooks. Employees frequently spend significant time manually locating the relevant information. The objective of this project is to reduce that friction by building an intelligent assistant that can understand a natural-language question and retrieve the correct information from the organization's knowledge base.

The system must therefore support:

- ingestion of diverse internal documents,
- extraction of meaningful content from those documents,
- semantic indexing for retrieval,
- answer generation grounded in retrieved evidence,
- and transparent source attribution.

3. Design Objectives

The system was designed with the following goals in mind:

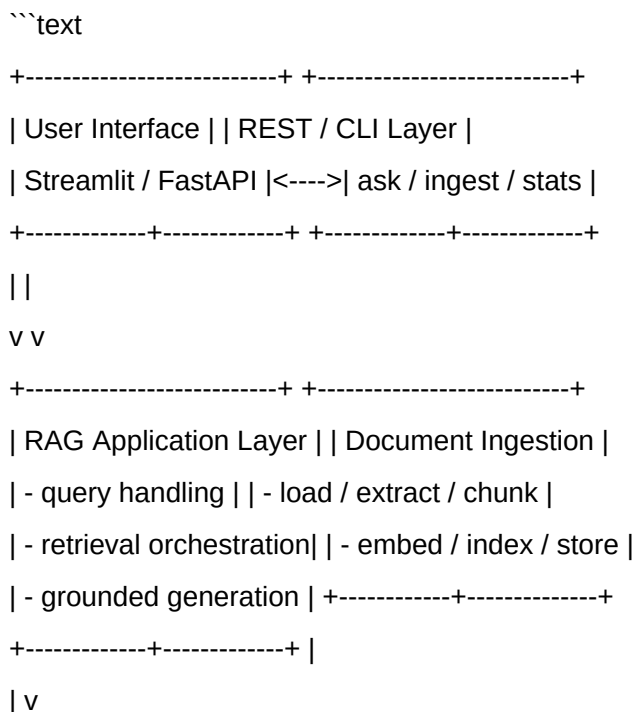
1. Accuracy and grounding: answers should be based on the indexed knowledge base rather than unsupported model memory.
2. Privacy: the system should operate locally without depending on external LLM or embedding APIs.
3. Simplicity and maintainability: the implementation should be modular and understandable.
4. Practicality: the solution should be usable through a Streamlit interface, a REST API, and a CLI.
5. Extensibility: the architecture should allow future improvements such as re-ranking, authentication, and deployment orchestration.

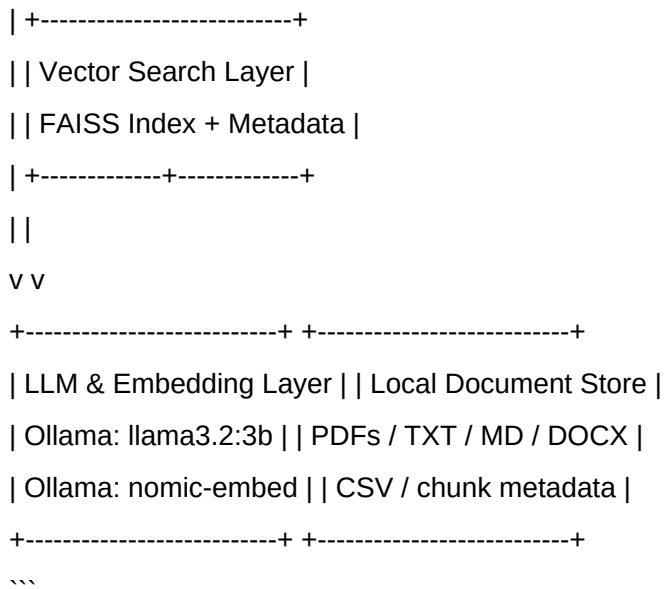
4. System Architecture

The system follows a modular RAG architecture composed of four main layers:

- Interface Layer: Streamlit, FastAPI, and CLI
- Application Layer: ingestion, retrieval, and generation logic
- Search and Storage Layer: FAISS vector store with metadata persistence
- Model Layer: Ollama-based LLM and embedding models

4.1 Architecture Diagram





4.2 Architectural Summary

The pipeline begins when a user submits a question or uploads a document. The system processes the document into chunks, generates embeddings, stores them in a FAISS index, and later retrieves the best matching chunks for a question. These chunks are passed to the local LLM, which generates a grounded answer and cites the relevant source documents.

5. Core Components

5.1 Ingestion Module

The ingestion module is responsible for transforming raw documents into searchable knowledge chunks. It handles:

- file discovery,
- format-specific loading,
- text extraction,
- chunking with overlap,
- metadata preservation,
- embedding generation, and
- storage in the vector index.

The design prioritizes deterministic preprocessing and structured metadata so that retrieval quality remains high and answers can be traced back to their origin.

5.2 Retrieval Module

The retrieval module takes a user query, converts it into an embedding, and searches the vector store for the most relevant chunks. The algorithm combines:

- semantic search via embeddings,
- keyword-based relevance via BM25-style scoring,
- and Reciprocal Rank Fusion (RRF) to merge both ranking signals.

This hybrid strategy is particularly effective for enterprise queries that may contain both abstract concepts and specific terms such as policy numbers, document names, or technical acronyms.

5.3 Generation Module

The generation module uses the retrieved context as grounding input for the LLM. The model is instructed to answer only from that context and to cite sources. This design is crucial for reducing hallucinations and improving the reliability of the assistant in enterprise settings.

5.4 Storage Module

The storage module uses FAISS as the vector index. Each stored chunk is associated with metadata including:

- source document name,
- page number,
- chunk index,
- and file hash for deduplication.

This enables precise citations and supports future extensions such as document-level filtering and permissions.

6. Data Flow

6.1 Document Ingestion Flow

1. The user uploads one or more documents.
2. The ingestion pipeline detects the file type.
3. The document is parsed into text.

4. The text is split into chunks with overlap.
5. Each chunk is embedded using an Ollama embedding model.
6. The chunk embeddings and metadata are stored in the FAISS index.

6.2 Query Flow

1. The user submits a natural-language question.
2. The query is embedded using the same embedding model.
3. The vector store retrieves candidate chunks.
4. Candidate chunks are re-scored using keyword-based relevance.
5. The highest-quality context is passed to the LLM.
6. The model produces a concise answer with citations.
7. The answer, references, and confidence score are returned to the user.

7. Technical Design Choices

7.1 Local-first Deployment

The system is implemented to run without external API dependencies. Ollama provides local LLM and embedding inference, ensuring privacy and independence from cloud services.

7.2 Choice of LLM

The default model is llama3.2:3b, selected for its balance of speed, quality, and local execution feasibility. The architecture is flexible enough to switch to other locally available models if needed.

7.3 Choice of Embedding Model

The embedding model is nomic-embed-text, chosen for its retrieval-oriented performance and local runtime support. This makes semantic search practical without external embedding services.

7.4 Choice of Vector Store

FAISS was selected because it is lightweight, efficient, and does not require the dependency stack associated with some other vector databases. It is well suited for this project's scope and environment constraints.

7.5 Chunking Strategy

The system uses a recursive chunking approach to preserve semantic coherence. The chunk size and overlap are tuned to balance retrieval precision and contextual completeness.

7.6 Grounded Prompting

The prompt instructs the model to answer only from the retrieved context and to cite sources. This is essential for trustworthy enterprise use and directly supports hallucination prevention.

8. Retrieval Strategy

The retrieval strategy is designed to be robust for both conceptual and exact-match questions. A purely semantic system can miss precise terms, while a purely keyword-based system can fail to capture intent. The solution combines both approaches.

8.1 Semantic Retrieval

Semantic retrieval uses embeddings to identify chunks that are conceptually related to the user's query.

8.2 Keyword Retrieval

Keyword retrieval uses BM25-style scoring to emphasize exact terms and domain-specific vocabulary.

8.3 Reciprocal Rank Fusion

The system combines the ranked results from both retrieval methods using Reciprocal Rank Fusion. This provides a more stable and balanced retrieval outcome.

This hybrid retrieval strategy is one of the strongest technical aspects of the system and directly supports the project's evaluation criteria.

9. Prompt Engineering and Generation

Prompt design is central to the assistant's reliability. The generation prompt instructs the model to:

- answer strictly from the provided context,
- remain concise,
- cite the relevant source documents,
- and indicate when the knowledge base does not contain enough information.

This level of grounding is essential for enterprise applications where incorrect or fabricated answers can be costly.

10. User Interface and Interaction Modes

The system provides three interaction modes:

1. Streamlit web application for interactive chat and document upload
2. FastAPI API for integration with other systems
3. CLI for ingestion, querying, evaluation, and statistics

This multi-mode design improves usability and makes the project suitable for both demos and practical deployment scenarios.

11. Evaluation Approach

The project includes an evaluation workflow to assess retrieval quality and answer usefulness. The evaluation focuses on:

- factual accuracy,
- relevance of retrieved context,
- source citation quality,
- and hallucination avoidance.

A sample evaluation set includes direct factual questions, multi-step questions, cross-document reasoning, and unanswered questions. This provides a practical testing strategy aligned with the assignment's evaluation criteria.

12. Engineering Quality and Maintainability

The implementation is structured to promote maintainability and future expansion. The codebase is separated into clearly defined modules for ingestion, retrieval, vector storage, API access, and UI logic. Configuration is centralized, which simplifies experimentation with different models, chunk sizes, and retrieval thresholds.

This separation of concerns also makes it easier to extend the system with improvements such as re-ranking, authentication, monitoring, or containerized deployment.

13. Scalability and Future Enhancements

The current implementation is well suited for small to medium-sized internal knowledge bases. However, the architecture can be extended for larger deployments by introducing:

- re-ranking models for improved precision,
- more scalable vector databases such as Qdrant or Weaviate,
- asynchronous ingestion pipelines,
- role-based access control,
- and deployment through Docker or Kubernetes.

These enhancements would improve scalability and enterprise readiness without changing the core RAG architecture.

14. Security and Privacy Considerations

Since the system runs locally with Ollama, it provides strong privacy guarantees for confidential internal documents. No proprietary content needs to leave the workstation or local environment in order to generate answers.

Future work could add authentication, audit logging, and document-level access policies to further align the system with enterprise security requirements.

15. Conclusion

The Enterprise Knowledge Assistant demonstrates a complete and practical RAG system for enterprise knowledge retrieval. It combines local LLM inference, vector search, hybrid retrieval, grounded prompting, and source-based answer generation into a cohesive application that is technically sound, privacy-preserving, and suitable for assignment submission and real-world demonstration.